ML Algorithms

1. Linear Regression:

Linear Regression models the relationship between an input variable and a continuous output using a straight line.

For a single feature: $\hat{y} = wx + b$

- x: input feature.
- w: slope or coefficient.
- b: intercept.
- $\hat{y}$: predicted target.

The slope tells us how much the predicted output changes when the input increases by one unit. The intercept is the predicted output when $x = 0$.

For multiple features: $\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b$

Linear regression learns the coefficients by minimizing the sum of squared residuals.

A residual is the difference between an observed value and the predicted value: $e_i = y_i - \hat{y}_i$

The ordinary least-squares objective is: $J(w, b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

Minimizing this objective finds a line that fits the observations according to squared error.

Suppose we have three observations:

x	**Actual** y
1	6
2	9
3	10

The least-squares line is: $\hat{y} = 2x + 4.333$

For $x = 2$: $\hat{y} = 2(2) + 4.333 = 8.333$

The residual is: $e = 9 - 8.333 = 0.667$

The line doesn't pass through every observation; it minimizes the total squared error across the dataset.

The coefficient of determination, R^2, compares the residual error against the variation around the mean:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Here, $\bar{y}$ is the mean of the observed targets.

01_linear_regression.py

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. Generate reproducible data
rng = np.random.default_rng(1)
x = rng.uniform(0, 10, 60)
y = 2.5 * x + 4 + rng.normal(0, 2, 60)

# 2. Prepare the feature matrix
X = x.reshape(-1, 1)

# 3. Train the Linear Regression model
model = LinearRegression()
```

```
model.fit(X, y)

# 4. Get predictions for drawing the regression line
x_line = np.linspace(0, 10, 200)
X_line = x_line.reshape(-1, 1)
y_line = model.predict(X_line)

# 5. Calculate model performance
r2 = model.score(X, y)

print(f"Learned equation: y = {model.coef_[0]:.2f}x + "
      f"{model.intercept_:.2f}")
print(f"R-squared: {r2:.3f}")

# 6. Plot the observed data points
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color="steelblue", label="Observed data",
            alpha=0.8)

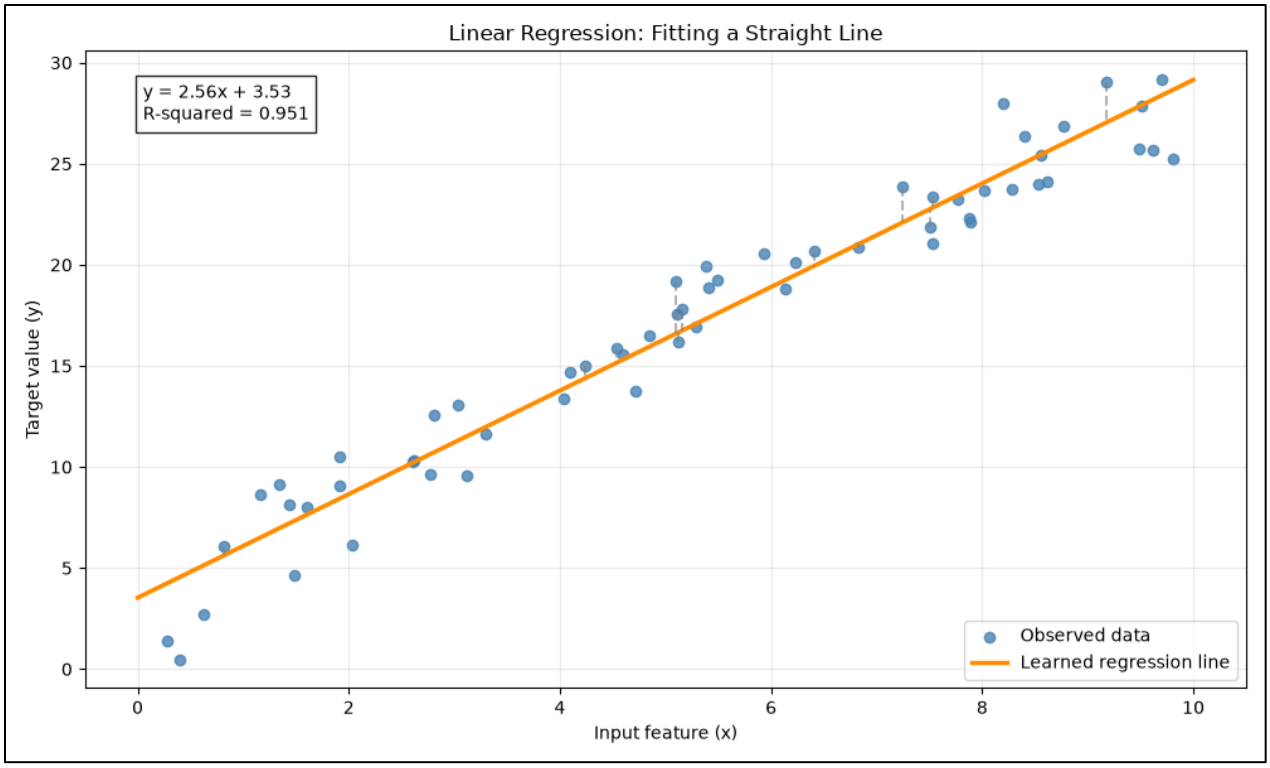
# 7. Plot the fitted regression line
plt.plot(x_line, y_line, color="darkorange", linewidth=2.5,
         label="Learned regression line")

# 8. Show residuals for a few observations
y_pred = model.predict(X)
for i in range(0, len(x), 5):
    plt.plot([x[i], x[i]], [y[i], y_pred[i]],
             color="gray", alpha=0.6, linestyle="--")

# 9. Add equation and R-squared to the graph
equation = (
    f"y = {model.coef_[0]:.2f}x + {model.intercept_:.2f}\n"
    f"R-squared = {r2:.3f}"
)
plt.text(0.05, 0.95, equation, transform=plt.gca().transAxes,
         verticalalignment="top",
         bbox=dict(facecolor="white", alpha=0.85))

# 10. Format and display the plot
plt.title("Linear Regression: Fitting a Straight Line")
plt.xlabel("Input feature (x)")
plt.ylabel("Target value (y)")
plt.legend()
plt.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 01_linear_regression.py
Learned equation: y = 2.56x + 3.53
R-squared: 0.951
```



2. Logistic Regression:

Despite its name, Logistic Regression is commonly used for classification rather than continuous-value regression. It first computes a linear score: $z = w^T x + b$

It then converts that score into a probability using the sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$

The output lies strictly between 0 and 1.

A common classification rule is: $\hat{y} = \begin{cases} 1, & P(y = 1 \mid x) \geq 0.5 \\ 0, & P(y = 1 \mid x) < 0.5 \end{cases}$

The threshold can be changed when false positives and false negatives have different costs.

Suppose a trained model has: $z = 1.2x - 2$
For $x = 3$: $z = 1.2(3) - 2 = 1.6$
Therefore: $P(y = 1 \mid x) = \frac{1}{1+e^{-1.6}} \approx 0.832$
At a threshold of 0.5, the model predicts Class 1.
For $x = 0$, the score is -2, so the estimated probability is approximately 0.119 and the predicted class is 0.

Logistic Regression typically minimizes binary cross-entropy, also called log loss:

$$J = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

Here $p_i = P(y_i = 1 \mid x_i)$.
The loss heavily penalizes confident predictions that are wrong. Regularization may also be applied to limit overly large coefficients.

The boundary at probability 0.5 occurs when: $\sigma(z) = 0.5 \Longrightarrow z = 0$
Thus: $w_1 x_1 + w_2 x_2 + b = 0$

02_logistic_regression.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Generate a two-class dataset
X, y = make_moons(
    n_samples=1000,
    noise=0.3,
    random_state=0
)

# 2. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

# 3. Create and train the Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Predict classes for unseen test data
y_pred = model.predict(X_test)

# 5. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print(f"Test accuracy: {accuracy:.2%}")
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))

# 6. Calculate class probabilities for the test samples
probabilities = model.predict_proba(X_test)
print("\nFirst 5 test probabilities:")
print(probabilities[:5].round(3))

# 7. Create a grid for visualizing the decision boundary
x1_min, x1_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
x2_min, x2_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))
prob_class1 = model.predict_proba(grid)[:, 1]
prob_class1 = prob_class1.reshape(xx.shape)

# 8. Plot the decision regions and training data
plt.figure(figsize=(10, 6))

plt.contourf(
    xx, yy, prob_class1,
    levels=[0, 0.5, 1],
```

```
    alpha=0.25
)

plt.contour(
    xx, yy, prob_class1,
    levels=[0.5],
    colors="black",
    linewidths=2
)

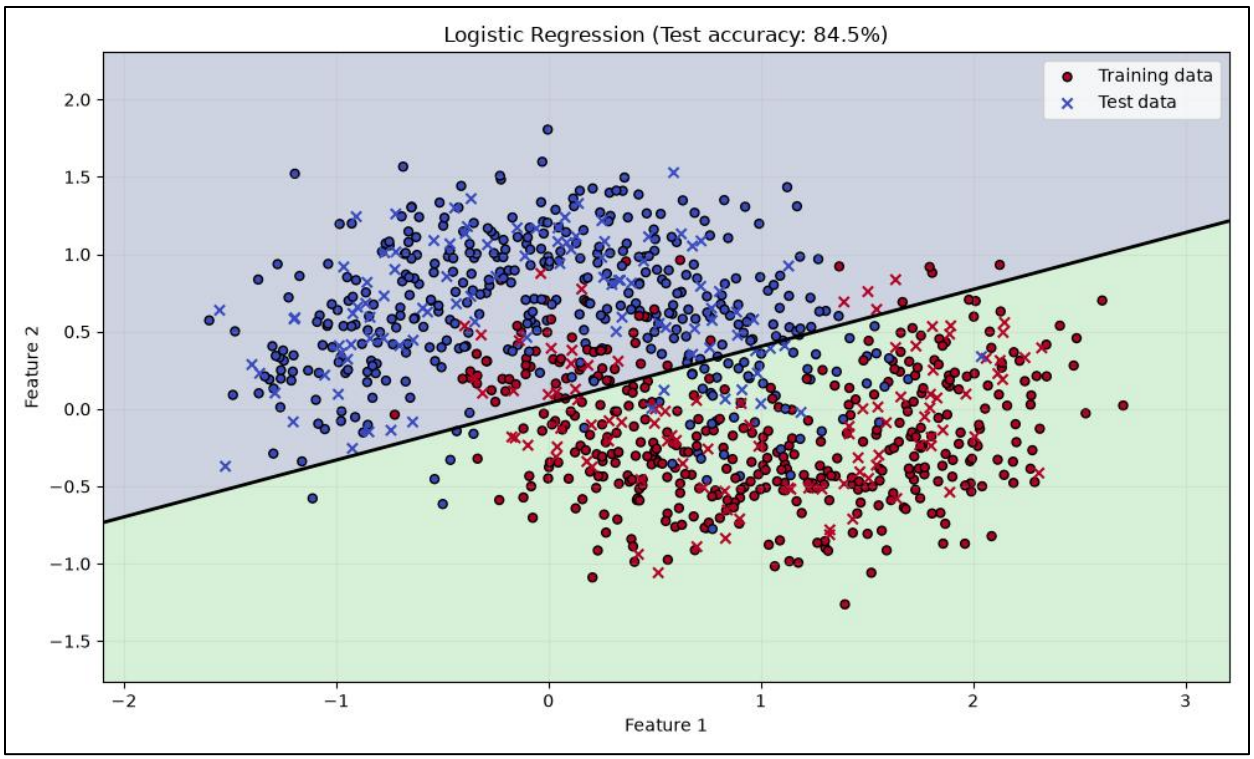
plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train, cmap="coolwarm",
    edgecolors="black", s=25,
    label="Training data"
)

plt.scatter(
    X_test[:, 0], X_test[:, 1],
    c=y_test, cmap="coolwarm",
    marker="x", s=35,
    label="Test data"
)

plt.title(f"Logistic Regression (Test accuracy: {accuracy:.1%})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 02_logistic_regression.py
Test accuracy: 84.50%
Confusion matrix:
[[89 11]
 [20 80]]

First 5 test probabilities:
[[0.966 0.034]
 [0.012 0.988]
 [0.955 0.045]
 [0.03  0.97 ]
 [0.081 0.919]]
```



3. K-Nearest Neighbors (KNN):

KNN predicts a new sample's class by examining the k nearest training samples.
For two feature vectors x and z, Euclidean distance is:

$$d(x,z) = \sqrt{\sum_{j=1}^{d}(x_j - z_j)^2}$$

The algorithm:

1. Computes distances from the new sample to training samples.
2. Selects the k nearest neighbors.
3. Counts their class labels.
4. Predicts the majority class.

In our implementation, $k = 15$.

Suppose a new point has coordinates $x = (2,3)$, and three training points are:

Point	Coordinates	Class	Distance
A	(1, 2)	0	$\sqrt{2} \approx 1.414$
B	(2, 4)	1	1
C	(4, 3)	1	2

If $k = 3$, two of the three neighbors belong to Class 1.
Therefore, the KNN prediction is Class 1.
A simple estimate of the probability of a class is the fraction of selected neighbors belonging to that class, although implementations can offer other weighting schemes.

Our program uses:

```
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=15)
)
```

Suppose one feature measures age from 18 to 70, while another measures income from 10,000 to 1,000,000. Without scaling, income may dominate Euclidean distance simply because its numerical scale is larger.
Standardization uses: $z_j = \frac{x_j - \mu_j}{\sigma_j}$
The pipeline fits the scaler on the training set and applies the learned transformation before KNN calculates distances.

Choosing k

- A very small k may produce a noisy boundary and overfit.
- A very large k creates smoother boundaries but may underfit.
- Odd values can reduce ties in binary classification, though they do not eliminate every possible tie.

03 knn.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Generate a two-class dataset
X, y = make_moons(
    n_samples=1000,
    noise=0.3,
    random_state=0
)

# 2. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

# 3. Create the KNN model with feature scaling
k = 15

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=k)
)

# 4. Train the model
model.fit(X_train, y_train)

# 5. Predict on unseen test samples
y_pred = model.predict(X_test)

# 6. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print(f"Number of neighbors (k): {k}")
print(f"Test accuracy: {accuracy:.2%}")
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
# 7. Create a grid for plotting the decision boundary
x1_min, x1_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
x2_min, x2_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))

# 8. Predict the class at every grid location
Z = model.predict(grid).reshape(xx.shape)

# 9. Plot decision regions and samples
plt.figure(figsize=(10, 6))

plt.contourf(
    xx, yy, Z,
    levels=[-0.5, 0.5, 1.5],
    cmap="coolwarm",
    alpha=0.25
)

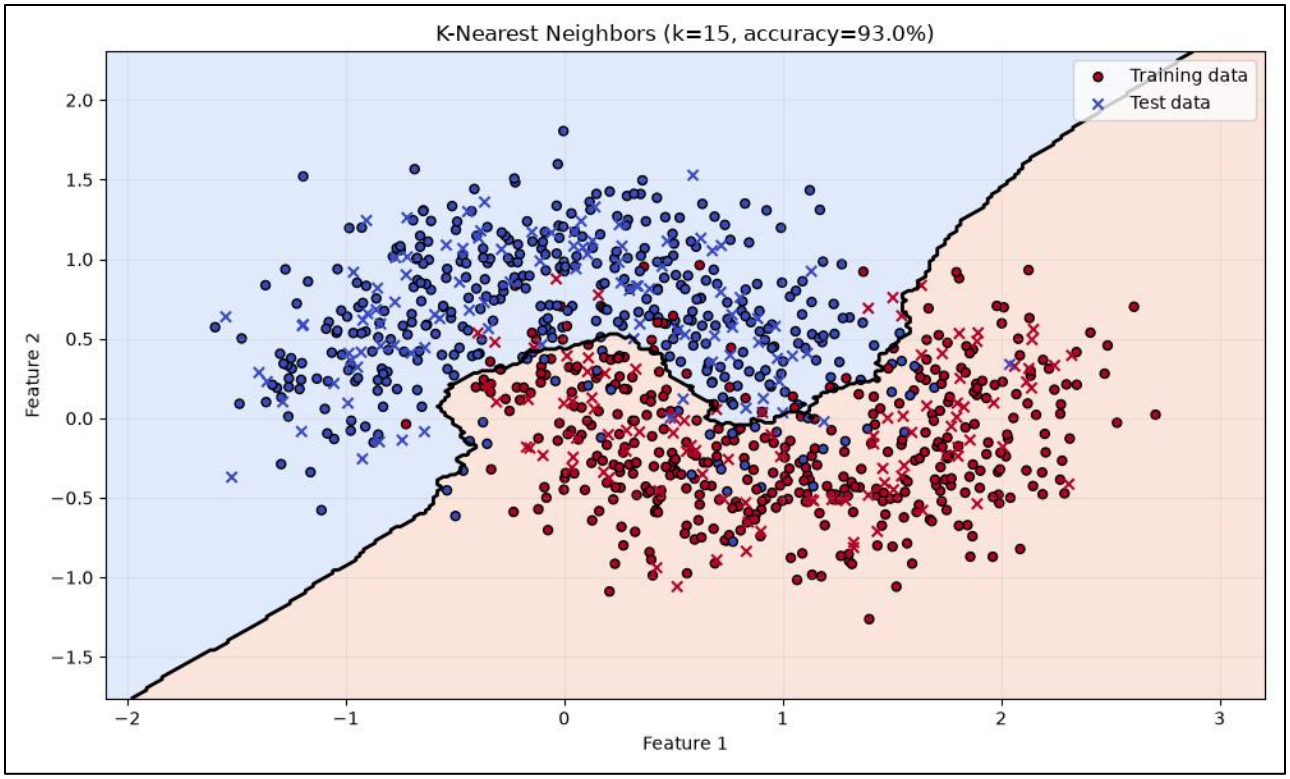
plt.contour(
    xx, yy, Z,
    levels=[0.5],
    colors="black",
    linewidths=2
)

plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train, cmap="coolwarm",
    edgecolors="black", s=25,
    label="Training data"
)

plt.scatter(
    X_test[:, 0], X_test[:, 1],
    c=y_test, cmap="coolwarm",
    marker="x", s=35,
    label="Test data"
)

plt.title(f"K-Nearest Neighbors (k={k}, accuracy={accuracy:.1%})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 03_knn.py
Number of neighbors (k): 15
Test accuracy: 93.00%
Confusion matrix:
[[96  4]
 [10 90]]
```



4. Decision Tree:

A Decision Tree learns a sequence of feature-based questions.
For example:

- Is Feature 2 greater than 0.4?
- If yes, is Feature 1 less than 1.2?
- Based on the answers, which class should be predicted?

The first node is the root. Internal nodes perform splits, branches represent outcomes, and leaf nodes provide predictions.

Our program uses:

```
DecisionTreeClassifier(max_depth=4, random_state=0)
```

The depth limit restricts how many successive decisions the tree can make along a path.

Gini impurity

For a node containing class proportions $p_1, \dots, p_C$, Gini impurity is:

$$G = 1 - \sum_{c=1}^{C} p_c^2$$

For a binary classification node: $G = 1 - p^2 - (1-p)^2 = 2p(1-p)$

- A pure node has $G = 0$.
- A binary node with equal class proportions has $G = 0.5$.

Numerical example

Suppose a node contains 10 samples: six from Class 0 and four from Class 1.

$$G = 1 - (0.6)^2 - (0.4)^2 = 0.48$$

A possible split creates two child nodes:

- Left child: five Class 0 and one Class 1.
- Right child: one Class 0 and three Class 1.

Their impurities are:

$$G_L = 1 - \left(\frac{5}{6}\right)^2 - \left(\frac{1}{6}\right)^2 = \frac{5}{18} \approx 0.278$$

$$G_R = 1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 = 0.375$$

The weighted impurity after splitting is:

$$G_{\text{split}} = \frac{6}{10} G_L + \frac{4}{10} G_R \approx 0.317$$

The impurity reduction is:

$$\Delta G = 0.48 - 0.317 = 0.163$$

A larger impurity reduction indicates a more useful split under this criterion.

Entropy and information gain

Another splitting criterion is entropy:

$$H = -\sum_{c=1}^{C} p_c \log_2 p_c$$

Information gain is the reduction in weighted entropy after a split:

$$IG = H_{\text{parent}} - \sum_j \frac{n_j}{n} H_j$$

Our code uses the default Gini criterion, not entropy.

Overfitting and interpretation

A very deep tree can memorize training samples and perform poorly on new data. Limiting depth, requiring minimum samples per leaf, or pruning can reduce this risk.

04_decision_tree.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Generate a two-class dataset
X, y = make_moons(
    n_samples=1000,
    noise=0.3,
    random_state=0
)

# 2. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

# 3. Create the Decision Tree model
model = DecisionTreeClassifier(
    max_depth=4,
    random_state=0
```

```
)

# 4. Train the model
model.fit(X_train, y_train)

# 5. Predict on unseen test data
y_pred = model.predict(X_test)

# 6. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print(f"Test accuracy: {accuracy:.2%}")
print(f"Tree depth: {model.get_depth()}")
print(f"Number of leaves: {model.get_n_leaves()}")

print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))

# 7. Create a grid for the decision-boundary plot
x1_min, x1_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
x2_min, x2_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))
Z = model.predict(grid).reshape(xx.shape)

# 8. Plot decision regions and samples
plt.figure(figsize=(10, 6))

plt.contourf(
    xx, yy, Z,
    levels=[-0.5, 0.5, 1.5],
    cmap="coolwarm",
    alpha=0.25
)

plt.contour(
    xx, yy, Z,
    levels=[0.5],
    colors="black",
    linewidths=2
)

plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train, cmap="coolwarm",
    edgecolors="black", s=25,
    label="Training data"
)

plt.scatter(
    X_test[:, 0], X_test[:, 1],
    c=y_test, cmap="coolwarm",
    marker="x", s=35,
    label="Test data"
)

plt.title(f"Decision Tree (max_depth=4, accuracy={accuracy:.1%})")
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()

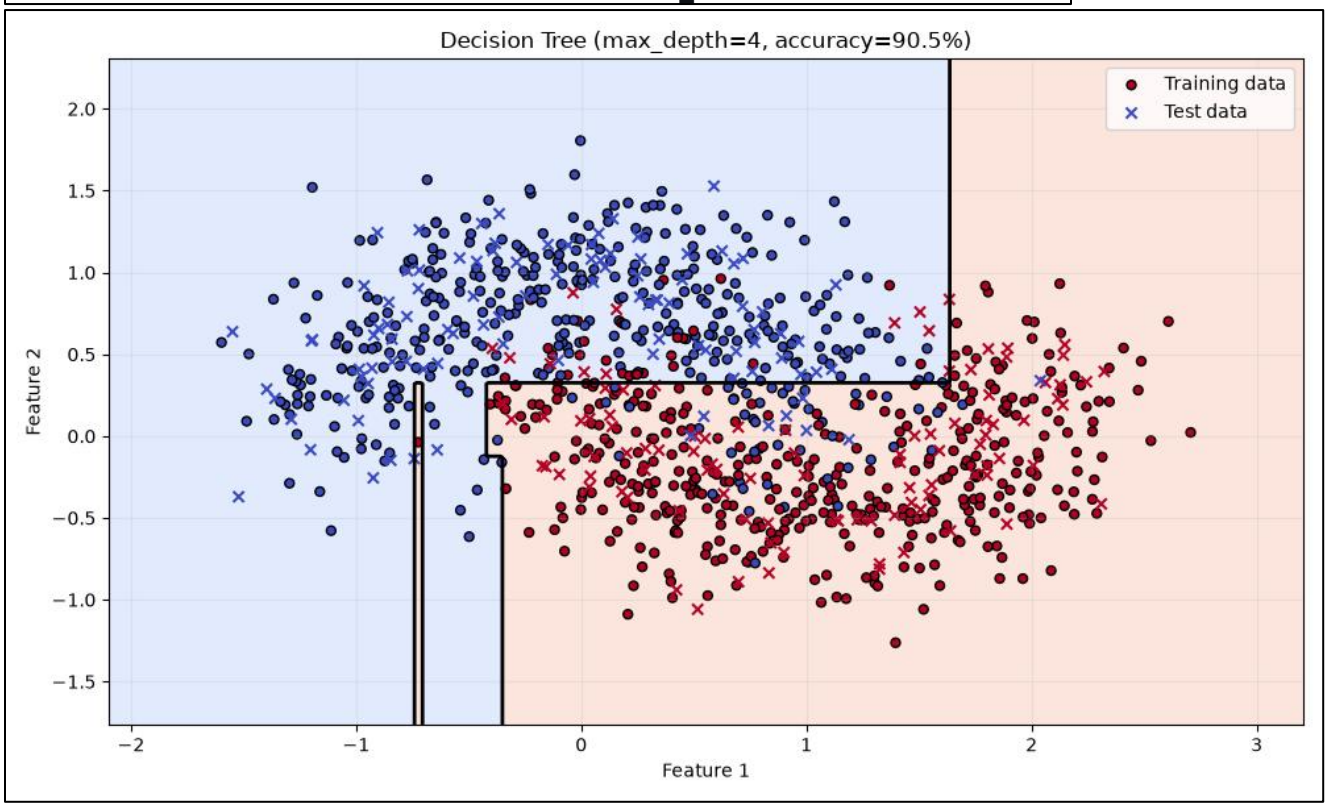
# 9. Visualize the learned decision tree
plt.figure(figsize=(18, 9))

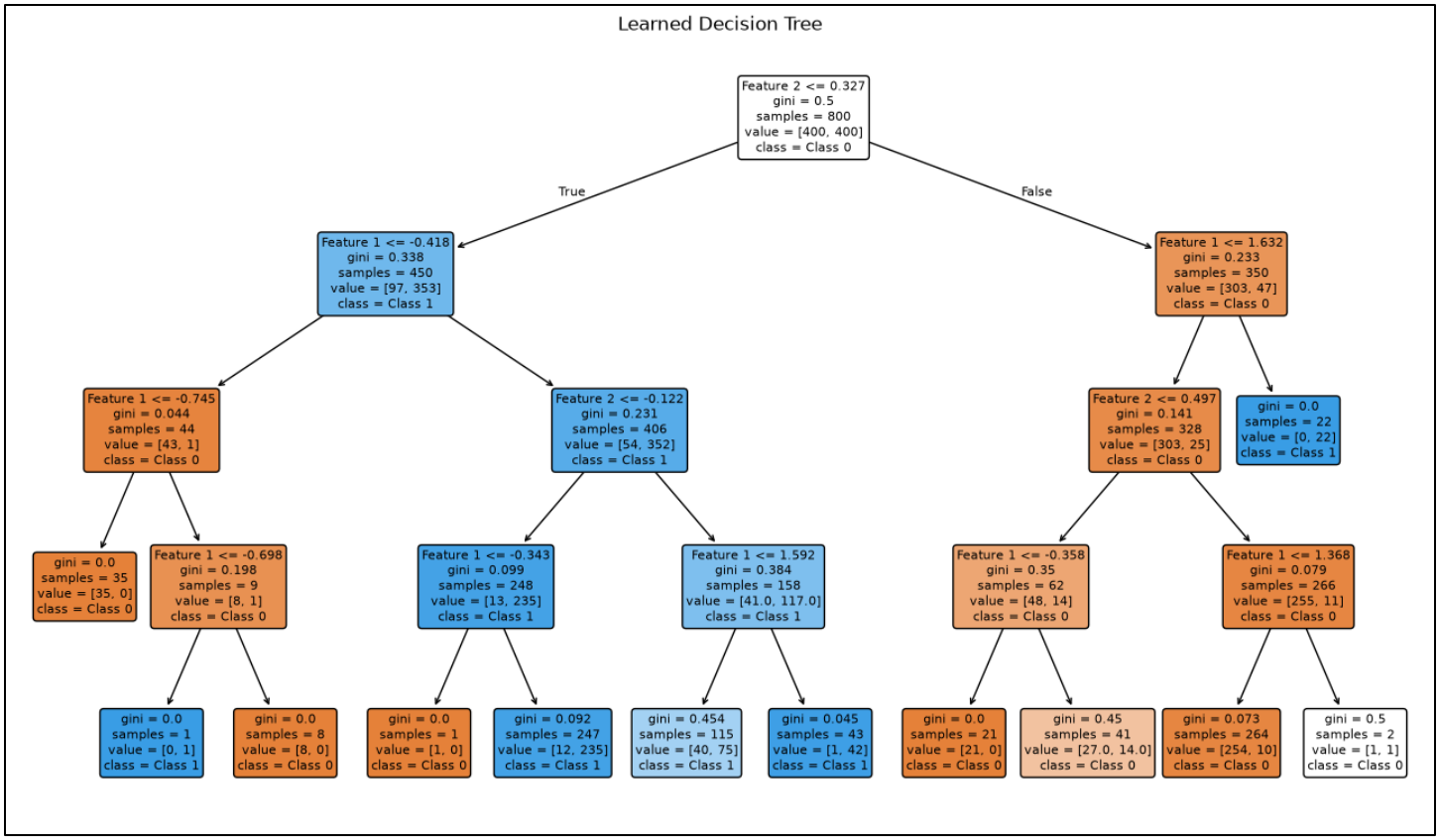
plot_tree(
    model,
    feature_names=["Feature 1", "Feature 2"],
    class_names=["Class 0", "Class 1"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Learned Decision Tree")
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 04_decision_tree.py
Test accuracy: 90.50%
Tree depth: 4
Number of leaves: 12

Confusion matrix:
[[91  9]
 [10 90]]
```





5. Random Forest:

Random Forest is an ensemble learning algorithm. Instead of relying on one Decision Tree, it trains many trees and combines their predictions.
Our implementation uses:

```
RandomForestClassifier(
    n_estimators=300,
    random_state=0,
    n_jobs=-1
)
```

The model trains 300 trees. Each tree uses a bootstrap sample of the training data, and candidate splits consider a random subset of features. These sources of randomness help diversify the trees.
For classification, the forest combines tree predictions through majority voting. Equivalently, class probabilities can be averaged across trees and converted into a class prediction.

Mathematical formulation
Let $h_t(x)$ be the class predicted by tree t. For binary or multiclass classification, the majority-vote prediction is:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \ldots, h_T(x)\}$$

A probability-based formulation is:

$$\hat{P}(y = c \mid x) = \frac{1}{T}\sum_{t=1}^{T} \hat{P}_t\,(y = c \mid x)$$

where T is the number of trees.

Worked numerical example
Suppose five trees classify a sample as follows:

Tree	Prediction
1	Class 0

2	Class 1
3	Class 1
4	Class 1
5	Class 0

Class 0 receives two votes and Class 1 receives three.
Therefore: $\hat{y} = \text{Class 1}$
The vote fraction for Class 1 is $3/5 = 0.6$. This is a vote fraction, not necessarily a perfectly calibrated probability.

Out-of-bag error and feature importance
Each tree's bootstrap sample leaves out some training observations. These are called out-of-bag observations and can be used to estimate generalization performance without a separate validation set, if enabled.
Our code also uses:
`importance = model.feature_importances_`
The default feature importance measures the total weighted impurity reduction attributable to each feature across the forest, normalized to sum to one. It is useful for exploration, but it can favor features with many possible split points and does not establish causality.

05_random_forest.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Generate a two-class dataset
X, y = make_moons(
    n_samples=1000,
    noise=0.3,
    random_state=0
)

# 2. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

# 3. Create the Random Forest model
model = RandomForestClassifier(
    n_estimators=300,
    random_state=0,
    n_jobs=-1
)

# 4. Train all trees in the forest
model.fit(X_train, y_train)

# 5. Predict on unseen test data
y_pred = model.predict(X_test)

# 6. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print(f"Number of trees: {len(model.estimators_)}")
print(f"Test accuracy: {accuracy:.2%}")
print(f"Training accuracy: {model.score(X_train, y_train):.2%}")
```

```
print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))

# 7. Create a grid for the decision-boundary plot
x1_min, x1_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
x2_min, x2_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))

# 8. Predict class probabilities across the grid
prob_class1 = model.predict_proba(grid)[:, 1]
prob_class1 = prob_class1.reshape(xx.shape)

# 9. Plot the ensemble decision boundary
plt.figure(figsize=(10, 6))

plt.contourf(
    xx, yy, prob_class1,
    levels=[0, 0.5, 1],
    cmap="coolwarm",
    alpha=0.25
)

plt.contour(
    xx, yy, prob_class1,
    levels=[0.5],
    colors="black",
    linewidths=2
)

plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train, cmap="coolwarm",
    edgecolors="black", s=25,
    label="Training data"
)

plt.scatter(
    X_test[:, 0], X_test[:, 1],
    c=y_test, cmap="coolwarm",
    marker="x", s=35,
    label="Test data"
)

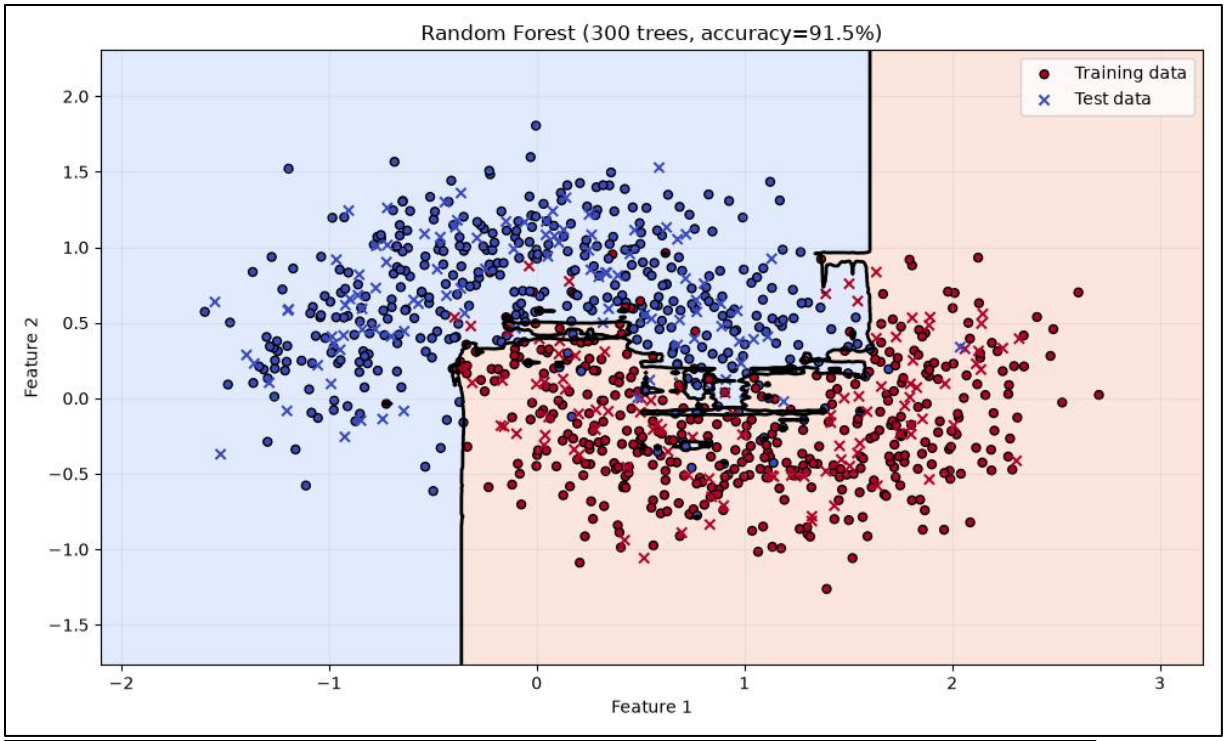
plt.title(f"Random Forest (300 trees, accuracy={accuracy:.1%})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()

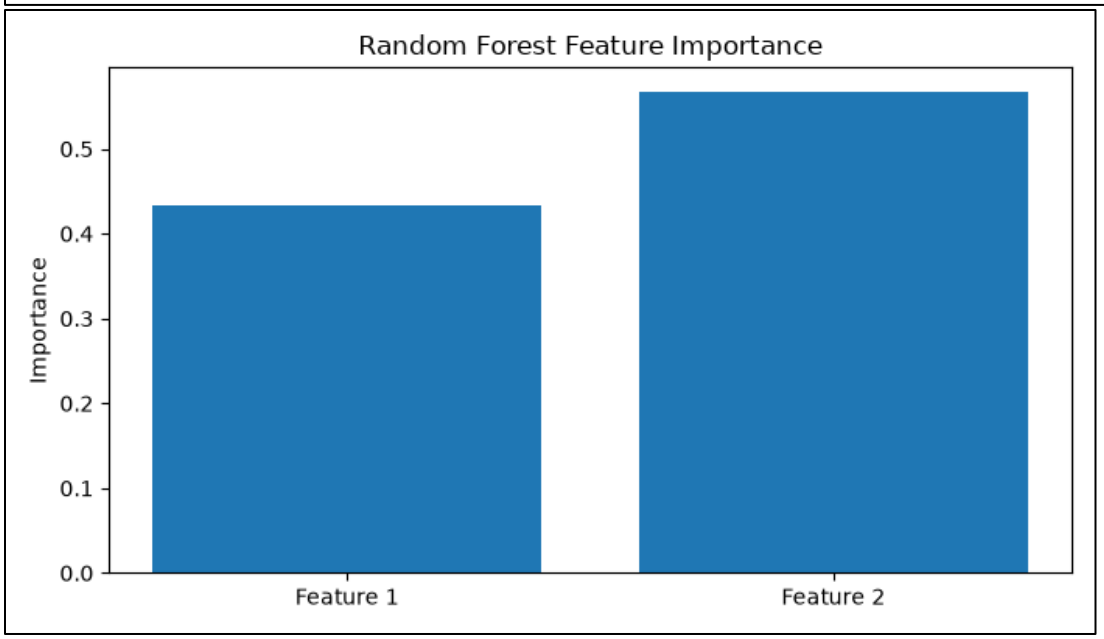
# 10. Plot feature importance
importance = model.feature_importances_
feature_names = ["Feature 1", "Feature 2"]

plt.figure(figsize=(7, 4))
plt.bar(feature_names, importance)
plt.ylabel("Importance")
plt.title("Random Forest Feature Importance")
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 05_random_forest.py
Number of trees: 300
Test accuracy: 91.50%
Training accuracy: 100.00%

Confusion matrix:
[[93  7]
 [10 90]]
```





6. Gradient Boosting:

Random Forest trains trees with substantial independence and combines them. Gradient Boosting builds an ensemble sequentially, where each new tree helps correct errors made by the existing ensemble.

Our program uses:

```
GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=2,
    random_state=0
)
```

This creates 100 boosting stages, with shallow trees of maximum depth 2.

The key parameters are:

- n_estimators: number of boosting stages.
- learning_rate: scales each tree's contribution.
- max_depth: controls the complexity of each tree.

Mathematical formulation

Gradient Boosting can be understood as stage-wise optimization of a loss function.
Starting with an initial model $F_0(x)$, the ensemble is updated as:

$$\boxed{F_m(x) = F_{m-1}(x) + \eta h_m(x)}$$

Here:

- $F_m(x)$ is the ensemble after stage m.
- $h_m(x)$ is the newly fitted tree's contribution.
- η is the learning rate.

More generally, the new learner is chosen to reduce the loss by approximating its negative gradient with respect to the current predictions.

For binary classification with logistic loss, the predicted probability is:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-F(x)}}$$

The boosting trees work on the evolving model scores rather than simply taking a majority vote from independently trained trees.

Worked numerical example

Consider a simplified numerical boosting example. Suppose the initial prediction is 0.50, the next tree's contribution is 0.20, and the learning rate is 0.10.
Then:

$$F_1 = 0.50 + 0.10(0.20) = 0.52$$

A second tree contributes \(-0.10\):

$$F_2 = 0.52 + 0.10(-0.10) = 0.51$$

This demonstrates the additive update mechanism. In actual classification boosting, updates typically operate on a model score or log-odds scale, so these example values are not intended as literal class probabilities.

Evaluation

Your program computes accuracy, confusion matrix, training accuracy, and log loss.
For predicted probabilities p_i, binary log loss is:

$$L = -\frac{1}{n}\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

Lower log loss means the model's predicted probabilities align better with the observed labels, with strong penalties for confident mistakes.

06_gradient_boosting.py

```
import numpy as np
```

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, log_loss

# 1. Generate a two-class dataset
X, y = make_moons(
    n_samples=1000,
    noise=0.3,
    random_state=0
)

# 2. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

# 3. Create the Gradient Boosting model
model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=2,
    random_state=0
)

# 4. Train the model
model.fit(X_train, y_train)

# 5. Predict on unseen test data
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)

# 6. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print(f"Number of boosting stages: {model.n_estimators}")
print(f"Test accuracy: {accuracy:.2%}")
print(f"Training accuracy: {model.score(X_train, y_train):.2%}")
print(f"Test log loss: {log_loss(y_test, y_prob):.4f}")

print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))

# 7. Create a grid for the decision-boundary plot
x1_min, x1_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
x2_min, x2_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))

# 8. Predict class probabilities across the grid
prob_class1 = model.predict_proba(grid)[:, 1]
prob_class1 = prob_class1.reshape(xx.shape)
```

```
# 9. Plot decision regions and samples
plt.figure(figsize=(10, 6))

plt.contourf(
    xx, yy, prob_class1,
    levels=[0, 0.5, 1],
    cmap="coolwarm",
    alpha=0.25
)

plt.contour(
    xx, yy, prob_class1,
    levels=[0.5],
    colors="black",
    linewidths=2
)

plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train, cmap="coolwarm",
    edgecolors="black", s=25,
    label="Training data"
)

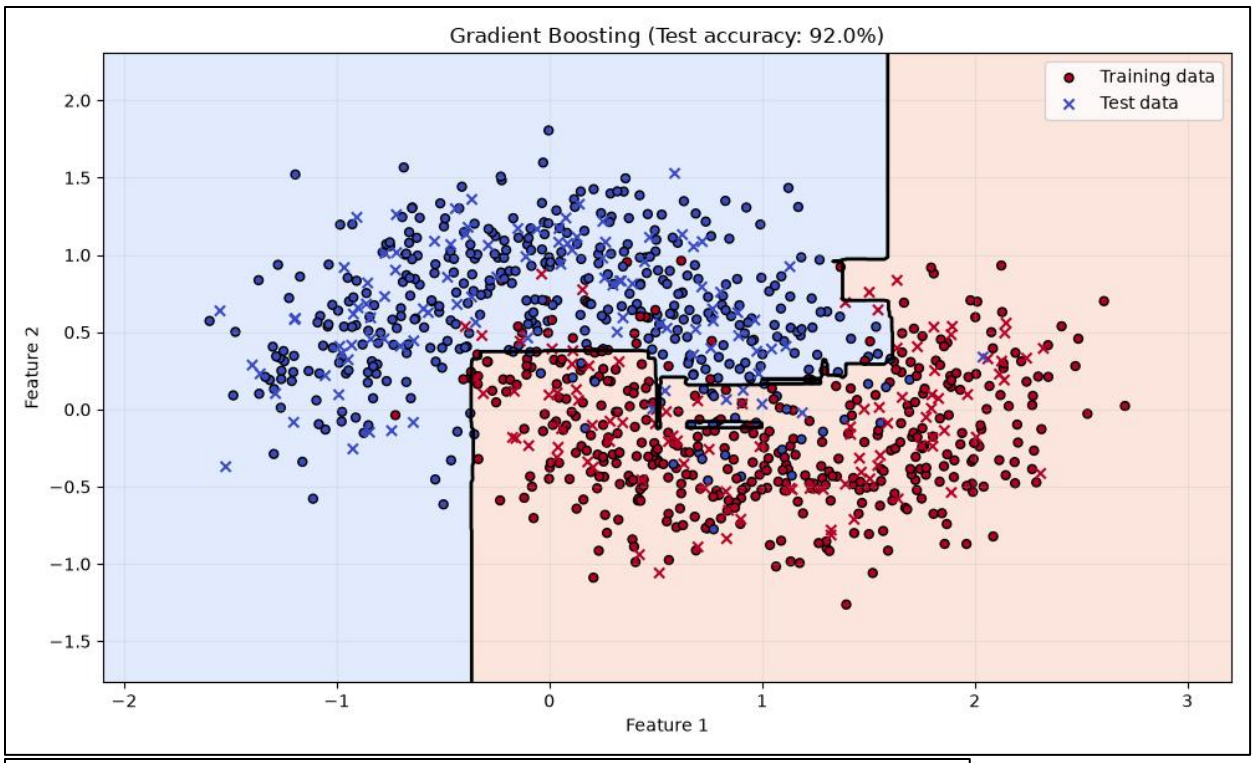
plt.scatter(
    X_test[:, 0], X_test[:, 1],
    c=y_test, cmap="coolwarm",
    marker="x", s=35,
    label="Test data"
)

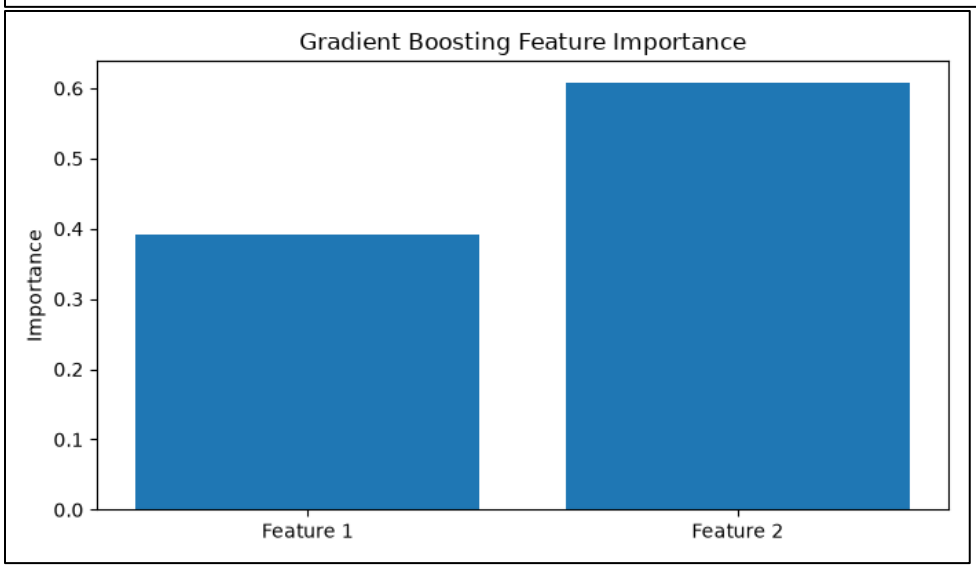
plt.title(f"Gradient Boosting (Test accuracy: {accuracy:.1%})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()

# 10. Plot feature importance
plt.figure(figsize=(7, 4))
plt.bar(
    ["Feature 1", "Feature 2"],
    model.feature_importances_
)
plt.ylabel("Importance")
plt.title("Gradient Boosting Feature Importance")
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 06_gradient_boosting.py
Number of boosting stages: 100
Test accuracy: 92.00%
Training accuracy: 93.38%
Test log loss: 0.1950

Confusion matrix:
[[93  7]
 [ 9 91]]
```





7. Support Vector Machine:

SVM searches for a decision boundary that separates classes with a large margin. The margin is the distance between the boundary and the closest relevant training points.
Those closest points are called support vectors. They play a central role in determining the boundary.

Our implementation uses:

```
model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0, gamma="scale")
)
```

This combines feature standardization with an SVM using the radial basis function (RBF) kernel.

Linear SVM formulas
For a linear classifier, the decision boundary is: $w^T x + b = 0$
The prediction is based on the sign of the decision function: $f(x) = w^T x + b$

The signed classification margin for a sample with label $y_i \in \{-1, +1\}$ is: $y_i(w^T x_i + b)$

The distance of a point to the boundary is: $d = \frac{|w^T x + b|}{\|w\|}$

For linearly separable data, the hard-margin optimization problem is: $\boxed{\min_{w,b} \frac{1}{2}\|w\|^2}$ subject to: $y_i(w^T x_i + b) \geq 1$

For data that is not perfectly separable, soft-margin SVM introduces slack variables and a penalty parameter C:

$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i$ subject to: $y_i(w^T x_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0$

A larger C penalizes training violations more strongly, while a smaller \(C\) permits more violations in exchange for a potentially smoother boundary.

The RBF kernel

The RBF kernel allows SVM to model nonlinear boundaries: $\boxed{K(x, z) = e^{-\gamma\|x-z\|^2}}$

- Small γ creates broader influence from each sample.
- Large γ creates more localized influence.
- C controls the trade-off between margin size and training errors.

The kernel effectively computes similarities in an implicit feature space without explicitly constructing all transformed features.

Worked numerical example

Suppose a linear SVM has: $w = (2,1), \qquad b = -3$

For the sample $x = (2,1)$: $f(x) = 2(2) + 1(1) - 3 = 2$

The positive score predicts the positive class.

The distance to the boundary is: $d = \frac{|2|}{\sqrt{2^2+1^2}} = \frac{2}{\sqrt{5}} \approx 0.894$

For a nonlinear RBF SVM, the decision score is computed from the support vectors and their learned coefficients rather than this simple two-weight equation.

07_svm.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Generate a two-class dataset
X, y = make_moons(
    n_samples=1000,
    noise=0.3,
    random_state=0
)

# 2. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

# 3. Create a scaled RBF-kernel SVM
```

```
model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0, gamma="scale")
)

# 4. Train the model
model.fit(X_train, y_train)

# 5. Predict on unseen test data
y_pred = model.predict(X_test)

# 6. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print(f"Test accuracy: {accuracy:.2%}")
print(f"Training accuracy: {model.score(X_train, y_train):.2%}")
print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))

# 7. Access the trained SVM inside the pipeline
svm = model.named_steps["svc"]
scaler = model.named_steps["standardscaler"]

print(f"\nNumber of support vectors: {svm.n_support_}")
print(f"Total support vectors: {len(svm.support_)}")

# Convert support vectors back to original feature coordinates
support_vectors = scaler.inverse_transform(svm.support_vectors_)

# 8. Create a grid for the decision-boundary plot
x1_min, x1_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
x2_min, x2_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))

# 9. Calculate the decision function over the grid
decision_values = model.decision_function(grid)
decision_values = decision_values.reshape(xx.shape)

# 10. Plot decision regions, boundary, and support vectors
plt.figure(figsize=(10, 6))

plt.contourf(
    xx, yy, decision_values,
    levels=[-100, 0, 100],
    cmap="coolwarm",
    alpha=0.25
)

# Decision boundary: decision function = 0
plt.contour(
    xx, yy, decision_values,
    levels=[0],
    colors="black",
    linewidths=2
)

# Margin contours for the SVM decision function
```

```
plt.contour(
    xx, yy, decision_values,
    levels=[-1, 1],
    colors="gray",
    linestyles="--",
    linewidths=1.5
)

# Training and test samples
plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train, cmap="coolwarm",
    edgecolors="black", s=25,
    label="Training data"
)

plt.scatter(
    X_test[:, 0], X_test[:, 1],
    c=y_test, cmap="coolwarm",
    marker="x", s=35,
    label="Test data"
)

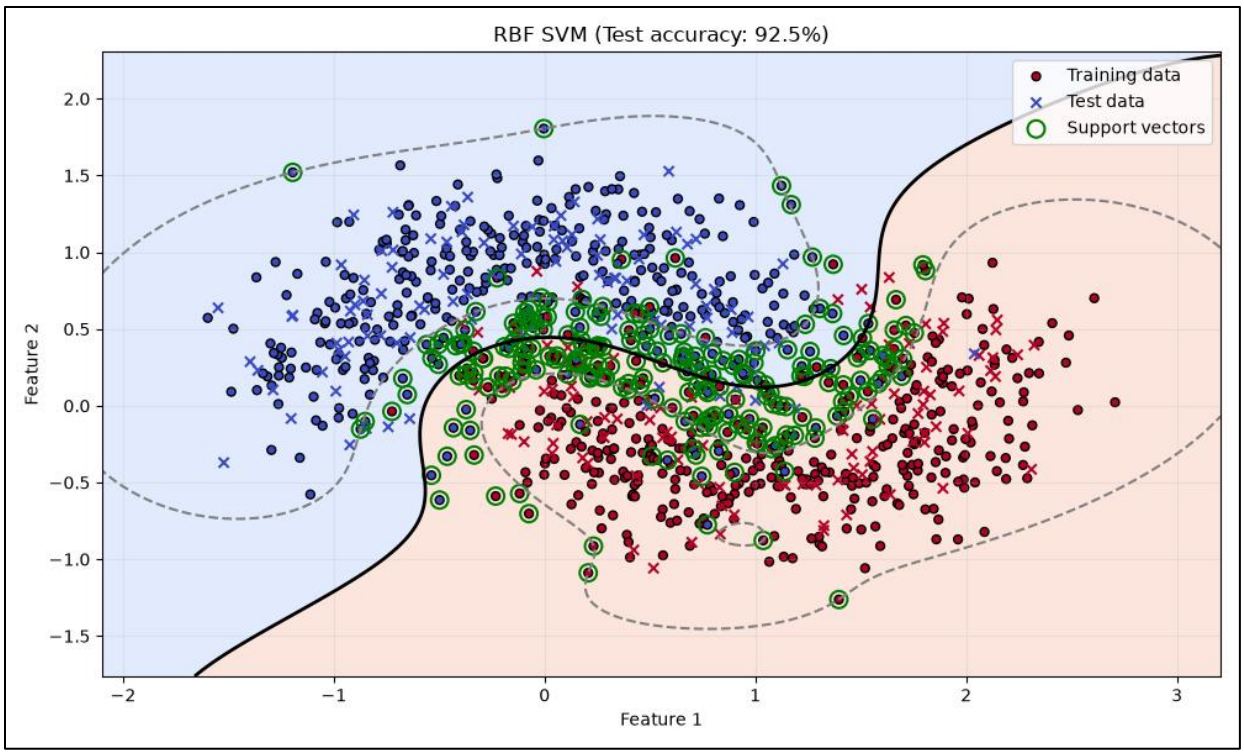
# Highlight support vectors
plt.scatter(
    support_vectors[:, 0], support_vectors[:, 1],
    s=100, facecolors="none", edgecolors="green",
    linewidths=1.5,
    label="Support vectors"
)

plt.title(f"RBF SVM (Test accuracy: {accuracy:.1%})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 07_svm.py
Test accuracy: 92.50%
Training accuracy: 90.62%

Confusion matrix:
[[93  7]
 [ 8 92]]

Number of support vectors: [115 113]
Total support vectors: 228
```



8. Naive Bayes:

Naive Bayes applies Bayes' theorem to estimate the probability of a class given observed features.

Bayes' theorem is: $\boxed{P(C \mid X) = \frac{P(X|C)P(C)}{P(X)}}$

Where:

- $P(C \mid X)$: posterior probability of the class.
- $P(X \mid C)$: likelihood of the features under that class.
- $P(C)$: prior class probability.
- $P(X)$: evidence shared by the candidate classes.

The naive assumption is that features are conditionally independent given the class: $P(X \mid C) \approx \prod_{j=1}^{d} P\left(x_j \mid C\right)$

Gaussian Naive Bayes models each continuous feature's class-conditional distribution using a Gaussian.

Gaussian likelihood

For feature x_j in class C:

$$\boxed{P\left(x_j \mid C\right) = \frac{1}{\sqrt{2\pi\sigma_{j,C}^2}} \exp\left(-\frac{\left(x_j - \mu_{j,C}\right)^2}{2\sigma_{j,C}^2}\right)}$$

The model estimates the mean $\mu_{j,C}$ and variance $\sigma_{j,C}^2$ for each feature and class.

It then combines these likelihoods with the class prior and predicts the class with the highest posterior probability.

Worked numerical example

Suppose two classes have equal priors: $P(C_0) = P(C_1) = 0.5$

For an observed sample, assume the feature likelihoods are: $P(X \mid C_0) = 0.10, \qquad P(X \mid C_1) = 0.40$

The unnormalized posterior scores are:

$$s_0 = 0.10(0.5) = 0.05$$
$$s_1 = 0.40(0.5) = 0.20$$

Normalizing:

$$P(C_1 \mid X) = \frac{0.20}{0.05 + 0.20} = 0.80$$

The model predicts Class 1 with a posterior probability of 80% in this illustrative example.

08_naive_bayes.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, log_loss

# 1. Generate a two-class dataset
X, y = make_moons(
    n_samples=1000,
    noise=0.3,
    random_state=0
)

# 2. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

# 3. Create the Gaussian Naive Bayes model
model = GaussianNB()

# 4. Train the model
model.fit(X_train, y_train)

# 5. Predict classes and probabilities
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)

# 6. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print(f"Test accuracy: {accuracy:.2%}")
print(f"Training accuracy: {model.score(X_train, y_train):.2%}")
print(f"Test log loss: {log_loss(y_test, y_prob):.4f}")

print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nFirst 5 predicted probability pairs:")
print(y_prob[:5].round(3))

# 7. Create a grid for the decision-boundary plot
x1_min, x1_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
x2_min, x2_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))

# 8. Calculate the probability of Class 1 across the grid
prob_class1 = model.predict_proba(grid)[:, 1]
```

```
prob_class1 = prob_class1.reshape(xx.shape)

# 9. Plot decision regions and samples
plt.figure(figsize=(10, 6))

plt.contourf(
    xx, yy, prob_class1,
    levels=[0, 0.5, 1],
    cmap="coolwarm",
    alpha=0.25
)

plt.contour(
    xx, yy, prob_class1,
    levels=[0.5],
    colors="black",
    linewidths=2
)

plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train, cmap="coolwarm",
    edgecolors="black", s=25,
    label="Training data"
)

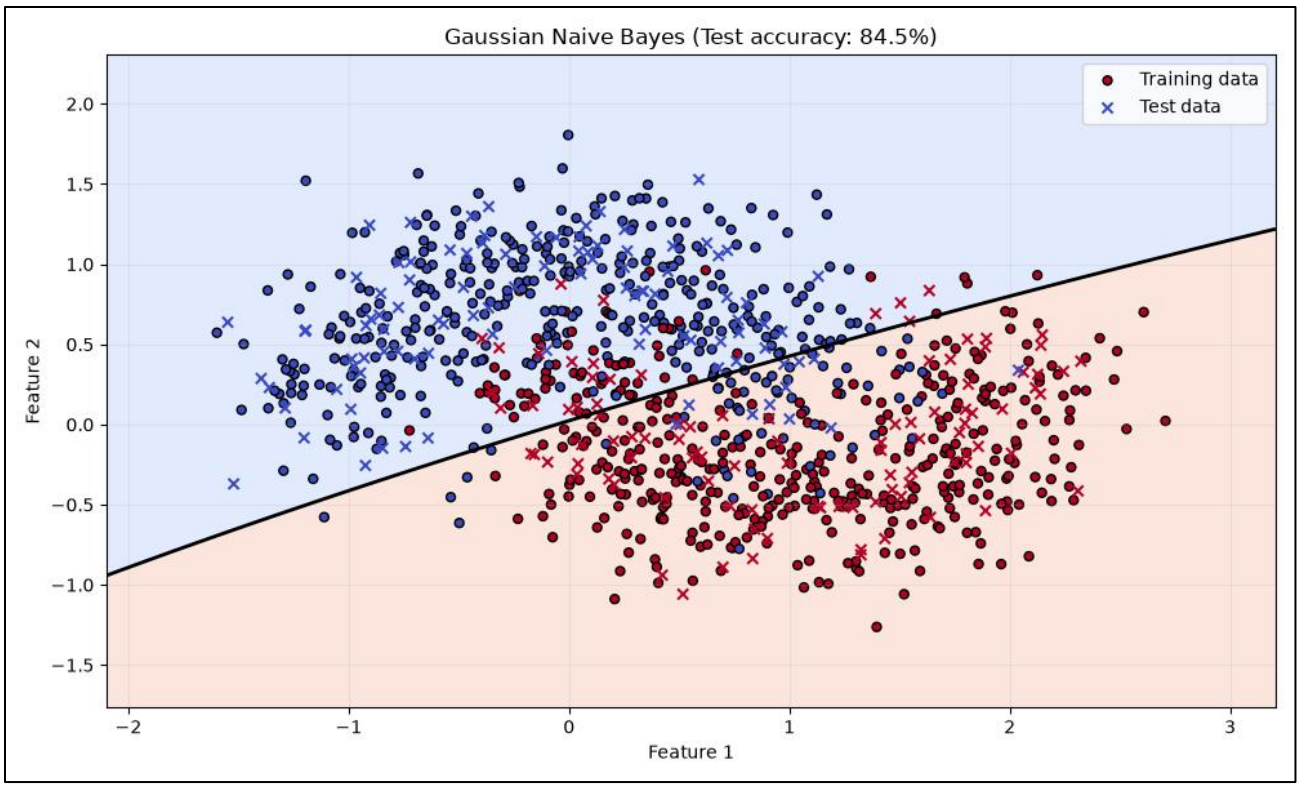
plt.scatter(
    X_test[:, 0], X_test[:, 1],
    c=y_test, cmap="coolwarm",
    marker="x", s=35,
    label="Test data"
)

plt.title(f"Gaussian Naive Bayes (Test accuracy: {accuracy:.1%})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 08_naive_bayes.py
Test accuracy: 84.50%
Training accuracy: 84.62%
Test log loss: 0.3243

Confusion matrix:
[[89 11]
 [20 80]]

First 5 predicted probability pairs:
[[0.988 0.012]
 [0.005 0.995]
 [0.983 0.017]
 [0.013 0.987]
 [0.041 0.959]]
```



9. K-Means Clustering:

K-Means is an unsupervised learning algorithm that partitions observations into K clusters. It tries to place similar points in the same cluster by minimizing their squared distances from their assigned cluster centers.

Our program uses:

```
KMeans(
    n_clusters=3,
    init="k-means++",
    n_init=10,
    random_state=42
)
```

Unlike the eight supervised algorithms, K-Means does not use target labels to learn its clusters.

Objective function

Suppose the dataset contains n points and K centroids. The K-Means objective is:

$$J = \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2$$

Here, c_i is the cluster assigned to point i, and μ_{c_i} is that cluster's centroid.

The algorithm alternates between two operations.

Assignment step:

$$c_i = \arg\min_{k \in \{1,\ldots,K\}} \|x_i - \mu_k\|^2$$

Assign each point to its nearest centroid.

Centroid update:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Replace each centroid with the mean of its assigned points.

These operations repeat until convergence or until the iteration limit is reached. K-Means generally converges to a local optimum, not necessarily the global optimum.

Worked numerical example

Consider four two-dimensional points:

$$A = (1,1), \quad B = (2,1), \quad C = (8,8), \quad D = (9,8)$$

Assume $K = 2$, with the initial centroids at A and C.

The nearby points are assigned as follows:

- Cluster 1: A, B
- Cluster 2: C, D

The new centroids are:

$$\mu_1 = \left(\frac{1+2}{2}, \frac{1+1}{2}\right) = (1.5,1)$$
$$\mu_2 = \left(\frac{8+9}{2}, \frac{8+8}{2}\right) = (8.5,8)$$

The sum of squared distances is:

$$J = (0.5)^2 + (0.5)^2 + (0.5)^2 + (0.5)^2 = 1$$

The clusters are well separated, so the objective is small.

Evaluation formulas

Our program prints inertia:

$$\text{Inertia} = \sum_i \left\| x_i - \mu_{c_i} \right\|^2$$

Lower inertia means tighter clusters for a given dataset. However, inertia cannot by itself determine the correct K, because it tends to decrease as the number of clusters increases.

We compute the silhouette score:

$$\boxed{s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}}$$

- $a(i)$: average distance from point i to other points in its own cluster.
- $b(i)$: the lowest average distance from point i to points in another cluster.

The silhouette score ranges from -1 to 1. Higher values generally indicate better-separated clusters.

09_kmeans.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Generate data with three natural groups
X, true_labels = make_blobs(
    n_samples=600,
    centers=3,
    cluster_std=1.0,
    random_state=42
)

# 2. Create the K-Means model
model = KMeans(
    n_clusters=3,
    init="k-means++",
    n_init=10,
    random_state=42
)
```

```
# 3. Discover clusters without providing labels
cluster_labels = model.fit_predict(X)

# 4. Extract the learned centroids
centroids = model.cluster_centers_

# 5. Evaluate the clustering
print(f"Number of clusters: {model.n_clusters}")
print(f"Number of iterations: {model.n_iter_}")
print(f"Inertia: {model.inertia_:.2f}")
print(f"Silhouette score: {silhouette_score(X, cluster_labels):.3f}")

# 6. Plot the discovered clusters
plt.figure(figsize=(10, 6))

plt.scatter(
    X[:, 0], X[:, 1],
    c=cluster_labels,
    cmap="viridis",
    s=35,
    alpha=0.8,
    label="Data points"
)

# 7. Plot the centroids
plt.scatter(
    centroids[:, 0], centroids[:, 1],
    marker="X",
    s=250,
    c="red",
    edgecolors="black",
    linewidths=1.5,
    label="Centroids"
)

plt.title("K-Means Clustering (K=3)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

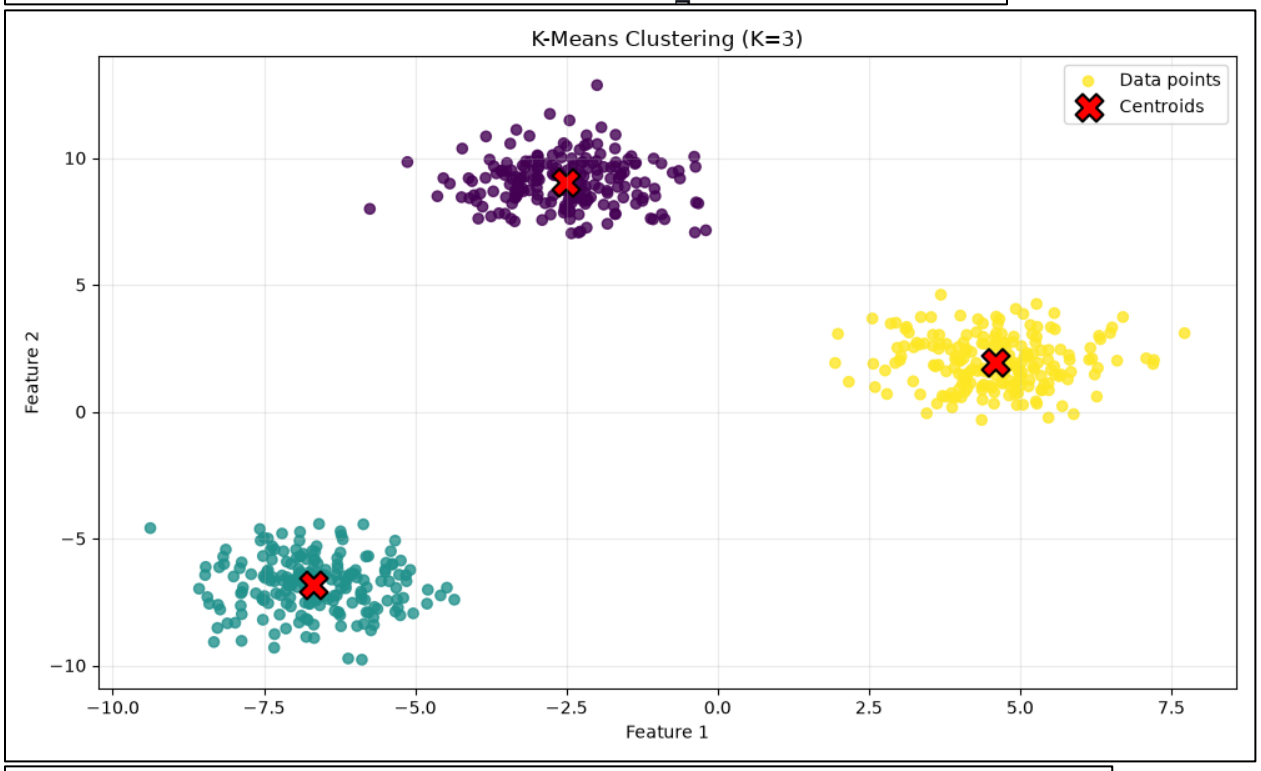
# 8. Plot the inertia for different values of K
k_values = range(1, 11)
inertias = []

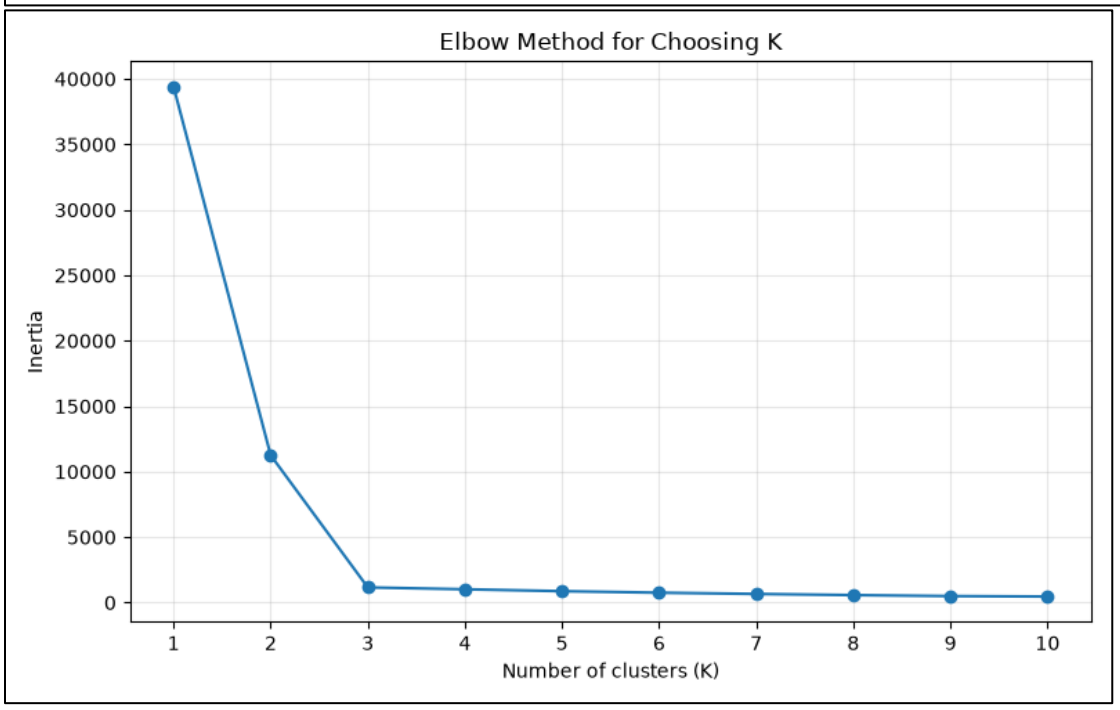
for k in k_values:
    candidate = KMeans(
        n_clusters=k,
        init="k-means++",
        n_init=10,
        random_state=42
    )
    candidate.fit(X)
    inertias.append(candidate.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(k_values, inertias, marker="o")
plt.xticks(list(k_values))
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Choosing K")
plt.grid(alpha=0.3)
plt.tight_layout()
```

```
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 09_kmeans.py
Number of clusters: 3
Number of iterations: 2
Inertia: 1160.90
Silhouette score: 0.843
```





10. Principal Component Analysis (PCA):

PCA reduces the dimensionality of data by finding orthogonal directions that capture the greatest variance. Your program starts with 10 features, standardizes them, and transforms the training and test samples into two principal components.

The new components are linear combinations of the original features. They are not necessarily two of the original columns.

Mathematical foundation

First, center the data: $X_c = X - \bar{X}$

For a centered dataset with n observations and d features, the sample covariance matrix is:

$$\boxed{S = \frac{1}{n-1} X_c^T X_c}$$

PCA finds the eigenvectors and eigenvalues of this covariance matrix:

$$Sv_j = \lambda_j v_j$$

- v_j: the direction of principal component j.
- λ_j: the variance along that direction.

The eigenvectors are ordered by decreasing eigenvalue:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$$

The first principal component captures the greatest variance, and each subsequent component captures the greatest remaining variance subject to orthogonality.

Projection and dimensionality reduction

If W_k contains the first k principal directions, the reduced representation is:

$$\boxed{Z = X_c W_k}$$

For your implementation, $k = 2$, so the original feature matrix with 10 columns becomes a transformed matrix with two columns.

You standardize the features before PCA using:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

This prevents features with larger numerical scales from dominating the variance calculation. The scaler is fitted only on the training data and then applied to the test data.

Explained variance ratio

The explained variance ratio for component $j$is:

$$\boxed{r_j = \frac{\lambda_j}{\sum_{\ell=1}^{d} \lambda_\ell}}$$

The cumulative explained variance after k components is:

$$R_k = \sum_{j=1}^{k} r_j$$

For example, suppose the first three components explain 40%, 25%, and 15% of the total variance.

Then:

$$R_3 = 0.40 + 0.25 + 0.15 = 0.80$$

The first three components preserve 80% of the variance. The remaining 20% lies in the discarded components.

Reconstruction and information loss

PCA can approximately reconstruct the original standardized data:

$$\hat{X}_c = ZW_k^T$$

The reconstruction error is commonly measured using mean squared error:

$$\boxed{\text{MSE} = \frac{1}{nd} \sum_{i=1}^{n} \sum_{j=1}^{d} \left(x_{ij} - \hat{x}_{ij}\right)^2}$$

Retaining more principal components generally reduces reconstruction error. If all components are retained, reconstruction is exact in ideal arithmetic for the full-rank representation.

10_pca.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Generate a dataset with 10 features
X, y = make_classification(
    n_samples=1000,
    n_features=10,
    n_informative=6,
    n_redundant=2,
    n_repeated=0,
    n_classes=2,
    random_state=42
)

# 2. Split the data before fitting preprocessing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 3. Standardize features using training data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Create PCA to retain two components
pca = PCA(n_components=2)

# 5. Learn components from training data
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# 6. Display explained variance
print("Original number of features:", X_train.shape[1])
print("Number of principal components:", X_train_pca.shape[1])

print("\nExplained variance ratio:")
print(pca.explained_variance_ratio_.round(4))

print(
    "Total variance retained:",
    f"{pca.explained_variance_ratio_.sum():.2%}"
)

# 7. Reconstruct test data approximately
X_test_reconstructed = pca.inverse_transform(X_test_pca)

reconstruction_mse = np.mean(
    (X_test_scaled - X_test_reconstructed) ** 2
)
```

```
print(f"Reconstruction MSE: {reconstruction_mse:.4f}")

# 8. Plot the training data in principal-component space
plt.figure(figsize=(9, 6))

plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=y_train,
    cmap="coolwarm",
    s=30,
    alpha=0.75
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA: 10 Features Reduced to 2")
plt.colorbar(label="Class label")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

# 9. Plot explained variance for all components
pca_full = PCA()
pca_full.fit(X_train_scaled)

individual_variance = pca_full.explained_variance_ratio_
cumulative_variance = np.cumsum(individual_variance)

components = np.arange(1, len(individual_variance) + 1)

plt.figure(figsize=(9, 5))

plt.bar(
    components,
    individual_variance,
    alpha=0.75,
    label="Individual explained variance"
)

plt.plot(
    components,
    cumulative_variance,
    marker="o",
    color="black",
    label="Cumulative explained variance"
)

plt.axhline(
    y=0.95,
    linestyle="--",
    label="95% variance reference"
)

plt.xticks(components)
plt.xlabel("Number of principal components")
plt.ylabel("Explained variance ratio")
plt.title("PCA Explained Variance")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\ML Algorithms> py 10_pca.py
Original number of features: 10
Number of principal components: 2

Explained variance ratio:
[0.2313 0.2008]
Total variance retained: 43.21%
Reconstruction MSE: 0.5675
```

